



Software Engineering II

Design and Architecture of an End-to-End Event Ticketing Platform

Project Definition

Course Instructor:
Dr. Gohari

Project Designers:
Hamed Ghoreyshi
Hossein Zareinejad

Spring 2026

1 Project Overview and High-Level Goals

The main goal of this project is to build a scalable and highly available online ticketing platform. The system will handle everything from event creation by organizers to real-time seat selection and secure payment processing for end-users. Our primary focus is to deliver a fair, transparent, and resilient solution. By leveraging distributed systems architecture, the platform must stay responsive and error-free even during high-concurrency traffic spikes when popular events go live.

1.1 Key Product Goals:

- **Seamless User Experience for Event Discovery and Booking:** Users should be able to quickly browse events and buy tickets without friction. The user interface needs to reflect live venue layouts, immediate seat availability, and dynamic pricing updates accurately.
- **Preventing Double-Booking and Mitigating Concurrency Issues:** The core booking engine must use advanced synchronization strategies to ensure no two users can purchase the same seat simultaneously. The platform should gracefully manage traffic bursts during high-demand sales to ensure fair distribution and avoid system crashes.
- **Reliable Transaction Management and Fault Tolerance:** We need a bullet-proof checkout workflow that securely processes payments and issues tickets immediately. It must also handle edge cases smoothly: if a transaction fails or a user times out, the temporarily locked seats must be automatically released back to the available pool.
- **De-coupled Architecture for Independent Scaling:** The system should be broken down into clean, isolated domains (such as catalog, reservation, payment, and notifications). This reduces inter-service dependency, accelerates development, and allows teams to scale heavy components independently based on live demand.

1.2 Organizational and Strategic Objectives:

- **Building Platform Trust and Reliability:** Delivering a highly dependable system that eliminates technical anxiety for major event organizers and guarantees a smooth, zero-downtime purchasing journey for buyers.
- **Empowering Organizers with Real-Time Analytics:** Providing organizers with dedicated dashboards that show real-time metrics on sales progress, revenue generation, and remaining seat capacities.
- **Keeping Users Informed via Instant Notifications:** Improving user engagement by sending transactional updates (via SMS or Email) for booking confirmations, immediate payment status changes, or event schedule updates.
- **Scalability for Future Market Expansion:** Ensuring the core infrastructure is flexible enough to smoothly handle future growth, adapt to new business domains, and process massive transaction volumes without requiring a complete rewrite.

2 System Capabilities and Core Deliverables

2.1 Functional Specifications:

- **Centralized Identity and Access Control:** Implementing a secure authentication mechanism (e.g., using JWT or OAuth2) with clear role-based permissions separating regular buyers from event organizers and system admins.
- **Venue Management and Event Lifecycle (Admin):** Admins must be able to configure venues with multi-sector layouts (rows, sections, and seat numbers) and map incoming events to specific time slots with custom pricing models.
- **Optimized Event Search and Filtering:** Providing end-users with an intuitive, highly optimized search engine to filter events by genre, date, location, and real-time availability.
- **High-Performance Seat Locking (Concurrency Control):** Building a robust locking mechanism that temporarily reserves a user's selected seats on the venue map, safely preventing race conditions during high-demand ticket releases.
- **Distributed Transaction Management and Rollback Patterns:** Handling the multi-step checkout workflow reliably. The system must orchestrate actions between the reservation engine and external payment gateways, using saga or compensation patterns to handle failed or aborted transactions cleanly.
- **Real-Time Status Updates and Ticket Issuance:** Utilizing web sockets and message brokers to stream order status changes live to the client, automatically generating a final ticket with a unique QR code upon successful payment.
- **Virtual Waiting Room and Traffic Throttling:** Designing an automated queueing system to throttle incoming traffic spikes during high-profile event drops, protecting core services from crashing.

2.2 Required System Documentation:

- **Risk Mitigation and Analysis Document:** A comprehensive guide identifying potential architectural failure points such as flash traffic spikes, payment gateway downtimes, and data synchronization bottlenecks along with clear engineering workarounds.
- **Product Vision Document:** A high-level guide defining the long-term goals of the platform, targeted user personas, and core business metrics to keep development aligned with market needs.

3 System Architecture and UML Modeling

In this section, you will document the system design using standard UML notation. All diagrams must be built using professional modeling tools and exported as high-quality, structured files (such as vector PDFs).

3.1 Required Design Diagrams:

1. Use Case Diagram:

- **Event Discovery:** Searching and filtering events based on tags, dates, and available capacity.
- **Booking Workflow:** Selecting seats dynamically from a map and triggering temporary locks.
- **Checkout & Issuance:** Processing payment securely and generating final ticket tokens.
- **Administrative Management:** Setting up venue layouts, pricing configurations, and scheduling constraints.

2. Class Diagram: Mapping the core data entities and their underlying business relationships:

- **Event:** Holds core event details, dates, classifications, and assigned organizer profiles.
- **Venue/Hall:** Defines physical dimensions, capacity constraints, and structural sectors.
- **Seat:** Tracks exact structural coordinates and real-time availability states (Available, Locked, Booked).
- **User:** Models system participants, profiles, and associated access permissions.
- **Reservation:** Tracks transient seat reservation states with dedicated TTL (Time-To-Live) expiration timers.
- **Payment:** Logs external transaction tokens, reference IDs, and accounting details.
- **Ticket:** The final verifiable token featuring embedded QR-code hashes and meta-data.

3. Sequence Diagram:

- **End-to-End Booking Flow:** Step-by-step visualization of a ticket purchase; covering the virtual waiting room entry, database lock application (e.g., via Redis), external payment redirection, and final success validation or timeout-triggered roll-back.
- **Asynchronous Notification Flow:** The sequence tracing how order states trigger event notifications through distributed message brokers.

4. Activity Diagram:

- **User Purchase Journey:** Modeling the operational choices a user makes from login to checkout, detailing the logical paths for successful payments, timeouts, or manual cancellations.
- **Admin Event Creation:** The sequential workflow for onboarding new venues, defining seat pricing categories, and pushing events live.

5. Component Diagram: Illustrating the high-level functional boundaries and key business domains of the system. Instead of prescribing a specific microservices breakdown, students must analyze the interactions between these core domains and propose their own decoupled architectural components:

- **Identity and Access Control Domain:** Handles registration, authentication, token management, and role-based checks.
- **Event Catalog and Discovery Domain:** Serving event metadata, search, and filtering requests.
- **Reservation Engine and Inventory Domain:** Manages transient seat states, inventory tracking, and concurrent lock allocation.
- **Billing and Checkout Domain:** Coordinates transactional workflows with external banking gateways.
- **Notification and Messaging Domain:** Dispatches transactional alerts, SMS confirmations, and ticket emails.
- **API Gateway Layer:** Acts as a unified edge interface for request routing, security compliance, and rate limiting.

6. Deployment Diagram:

- Visualizing container orchestration across Kubernetes clusters to enable automated auto-scaling during peak drops.
- Mapping relational databases (e.g., PostgreSQL for consistent billing) and in-memory datastores (e.g., Redis for transient locks).
- Showcasing the message broker topology (e.g., Kafka or RabbitMQ) driving asynchronous communication.

7. Automated Provisioning via IaC:

- Providing declarative Terraform scripts to provision cloud instances, load balancers, and managed databases automatically for a resilient environment.

4 Agile Lifecycle Management and Quality Assurance

Building a highly concurrent platform requires agile iterations and high software quality. Teams must implement Scrum methodologies and manage tracking through project boards like Jira.

4.1 Scrum Implementations and Delivery Milestones:

1. Product Backlog Management: Maintaining a cleanly organized and prioritized backlog featuring technical requirements like layout builders, queue processors, concurrency managers, and reporting panels.

2. Epics and Granular User Stories:

- **Epics** should capture high-level domain goals. Example: "Distributed Reservation Engine Setup".
- **User Stories** must define clear, incremental value. Example: "As a customer, I want my selected seat locked for 10 minutes so I can complete payment without losing it."

3. Sprint Planning and Execution: Selecting high-priority user stories for active sprints. Initial iterations should settle baseline database schemas and access controls, while subsequent sprints focus on queue workers and gateway integrations.

4. Core Development Best Practices: Writing maintainable codebases tracked on Git systems like GitLab. Changes should pass through clean Merge Request loops accompanied by thorough peer Code Reviews.

5. Sprint Reviews and Team Retrospectives:

- **Review Sessions:** Presenting functional increments to stakeholders (e.g., showing a live demo of concurrent request resolution).
- **Retrospectives:** Assessing workflow friction, looking at engineering bottlenecks discovered during the sprint, and planning process improvements.

6. QA Strategy and Verification Documentation: Since overbooking errors can break business trust, testing strategies must be exceptionally thorough:

- **Load and Stress Testing plans:** Writing test scripts to validate reservation integrity under artificial stress, simulating thousands of concurrent bot requests hitting the same seats.
- **Advanced Bug Hunting and Fault Analysis:** To guarantee complete business logic safety, teams are expected to run advanced testing techniques like Mutation Testing against core reservation code. Tracking explicit code coverage metrics inside transactional modules is mandatory.

4.2 Agile Operations:

- Setting up structured Jira boards with clean Epic-Task hierarchies.
- Generating precise Burndown Charts to analyze team velocity trends over time.
- Implementing automated GitLab CI/CD pipelines to run unit tests on every single commit.

5 Decoupled Services, Production Deployment, and Operations

Because ticketing platforms experience massive, sudden traffic swings, production stability is critical. In this module, teams will focus on service boundaries and error mitigation.

5.1 Isolating Service Boundaries (Bounded Contexts):

The engineering team must evaluate how core domains map out into cleanly decoupled architectural boundaries.

- **The Architectural Challenge:** Using Domain-Driven Design (DDD) principles to determine whether seat availability checks live inside the core event catalog domain or within an independent reservation context. Students are required to define their own microservice boundaries based on this analysis and demonstrate how structural coupling or circular dependencies have been eliminated.

5.2 Asynchronous Messaging Layouts:

Using distributed message brokers to decouple workflows, buffer unexpected traffic bursts, and ensure high reliability. Teams must trace how transaction outcomes pass asynchronously from billing workers to notification triggers.

5.3 Production Telemetry and Monitoring:

Mapping out deployment configurations on Kubernetes clusters to enable smooth horizontal auto-scaling and zero-downtime canary upgrades, combined with Prometheus and Grafana setups to actively monitor error rates, query latencies, and system health. This section requires architectural mapping only; no environment code needs to be written.

5.4 Incident Management Frameworks:

Defining clear workflows for handling live production issues. Teams must establish response matrices and on-call rotations to deal with service failures efficiently.

- **On-Call Rotations:** Assigning engineering roles to handle production issues immediately when alerts cross safety thresholds (such as increased latency or failing checkout calls).

5.5 Incident Postmortems (Blameless Root Cause Analysis):

Documenting and breaking down system outages after resolution to pinpoint root causes, map out business impacts, and implement preventative bug fixes.

- **Mandatory Postmortem Write-ups:**
 1. **Payment Processing Outages:** Analyzing what happens when external gateways fail or time out, leaving active seats stuck in locked limbo (including how compensation scripts fix this).

2. **Queue Worker Failures:** Documenting a scenario where the virtual waiting room system fails under immense stress, causing user calls to fall back to HTTP 503 errors.
3. **Inter-Service Network Partitioning:** Tracing failures when communication links between the edge gateway and internal engines break down unexpectedly.

6 Project Guidelines and Course Expectations

- **Simulating Team Roles:** Throughout the project timeline, team members must step into various roles such as Backend Developer, System Analyst, QA Specialist, or DevOps Engineer. Look at the architecture through each of these distinct perspectives.
- **Pragmatic Technical Design:** Ensure that your architectural models are realistic and practical, meaning your designs can be directly translated into production-ready infrastructure configurations.
- **Environment Portability:** All service code, database scripts, and docker configs must be cleanly packaged so any team member can pull the repository and spin up the complete environment locally without manual troubleshooting.
- **Adhering to Engineering Standards:** Every single artifact including documentation, API definitions, schemas, and diagrams must reflect modern software engineering standards.
- **Meticulous Technical Writing:** Maintain continuous, clear documentation covering schemas, test outcomes, and code patterns throughout the project lifecycle.
- **System Extensibility:** Ensure the platform architecture is open and extensible, allowing easy integration with external hooks, modern payment providers, and future mobile clients.
- **Mandatory Project Defense:** Every team member must be present during the live project presentation and architecture defense, ready to discuss design tradeoffs, justify technical choices, and walk through their code.

7 Grading and Submission Policy

7.1 Grading Breakdown:

- **Sections 1 to 4** are mandatory components of the project and account for **2 points** of the final grade.
- **Section 5** is considered the bonus assignment and accounts for an additional **2 points** of the final grade.

7.2 Submission Requirements:

- All project deliverables must be consolidated and uploaded as a single compressed **.zip** file.
- The submission package must include: the two required baseline documents (Risk Analysis and Product Vision), all standard UML diagrams, comprehensive Jira board exports, complete GitLab repository codebase directories, and all automated setup scripts or environment configurations associated with the project code.